

AI Agent 智能助手项目 · 面试讲稿

项目定位：面向企业内部的 **RAG + 工具调用 + 人工审批** Agent 后端服务 技术栈：FastAPI + LangGraph + LangChain + Chroma + SQLite + DeepSeek + 智谱 Embedding + Tavily

0. 电梯陈述（30 秒，开场必背）

II

这是一个企业内部的 AI 助手后端。用 FastAPI 提供 SSE 流式接口，LangGraph 编排 Agent 推理循环，DeepSeek 做推理、智谱 embedding-2 做向量化、Chroma 做本地知识库。

它支持四类能力：公司文档问答、联网搜索、数学计算，以及带人工审批的数据库查询。

最有意思的设计是数据库访问：我给模型暴露的是一个永远不会执行的占位工具，真正的 SQL 执行函数不在工具列表里。模型只能提交申请，图会 **interrupt** 挂起，前端把 SQL 展示给管理员，确认后用 **Command(resume)** 恢复执行、驳回就返回驳回文案——从结构上杜绝了模型绕过审批直接查库。

另外做了三层记忆，以及把工具调用全过程以事件流下发给前端，让用户能看见 AI 的思考过程。

1. 三分钟完整讲述（分段，可展开）

第一段：解决什么问题（30s）

企业内部问答有两类诉求：一类是**制度/手册类文档**问答，一类是**业务数据**查询。纯 LLM 答不了前者（没见过内部文档），直接把数据库交给模型又不安全。所以目标是：文档走 RAG，数据查询必须有人把关，同时整个过程要**流式可见**。

第二段：整体架构（60s）

- 接入层：FastAPI，两个路由模块 **/chat**（裸模型对话）和 **/rag**（文档管理 + Agent 问答）
- 编排层：LangGraph **StateGraph**，三个节点 + 条件路由 + 回环
- 能力层：4 个工具（知识库检索、联网搜索、计算器、SQL 申请）
- 记忆层：短期会话检查点、结构化用户画像、长期向量记忆
- 存储层：Chroma（知识库 / 记忆各一座）、SQLite（业务数据）、JSON（画像与文件元数据）
- 可观测：结构化 JSON 日志 + SSE 事件流

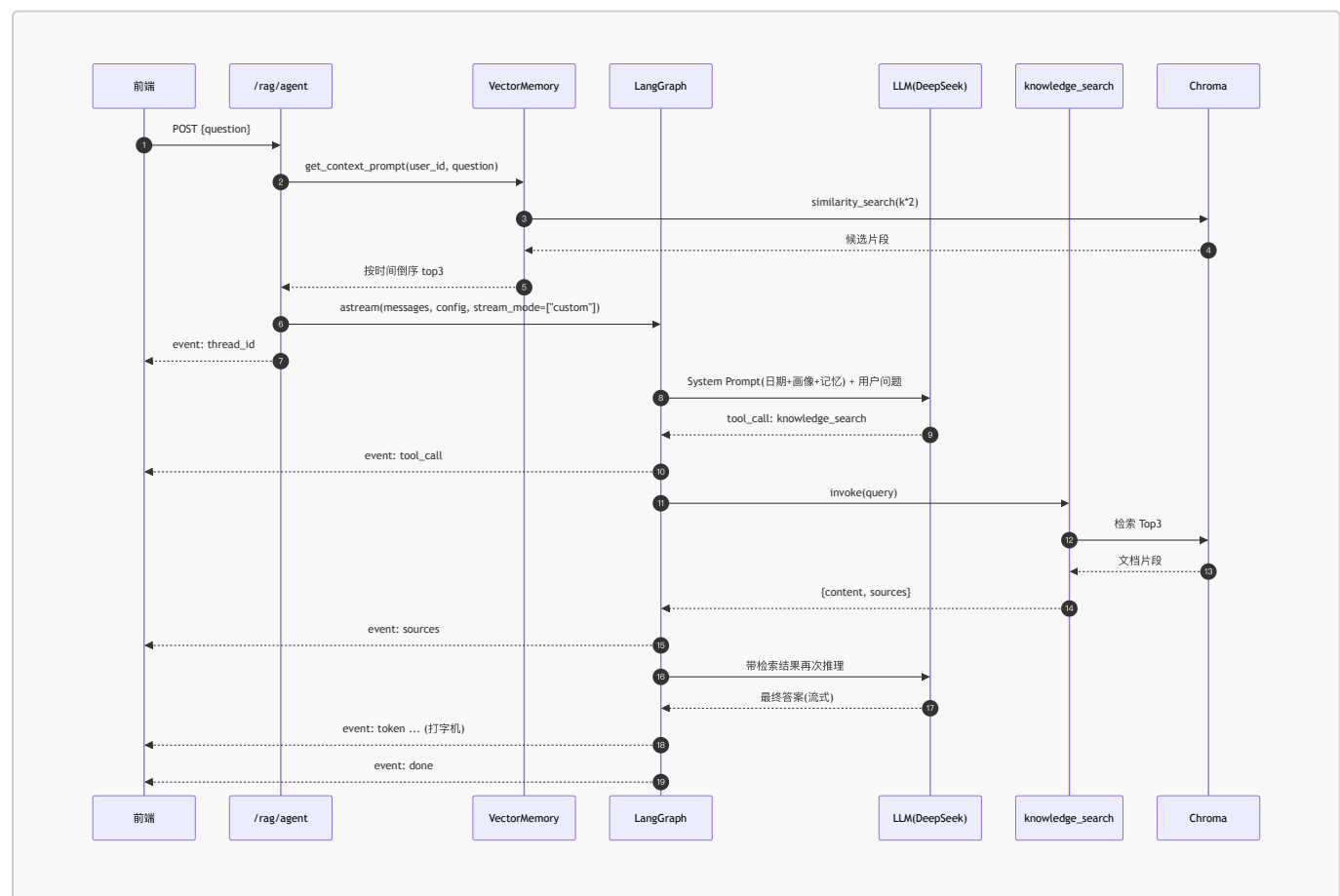
第三段：核心设计——人工审批（60s）

关键点讲三句：

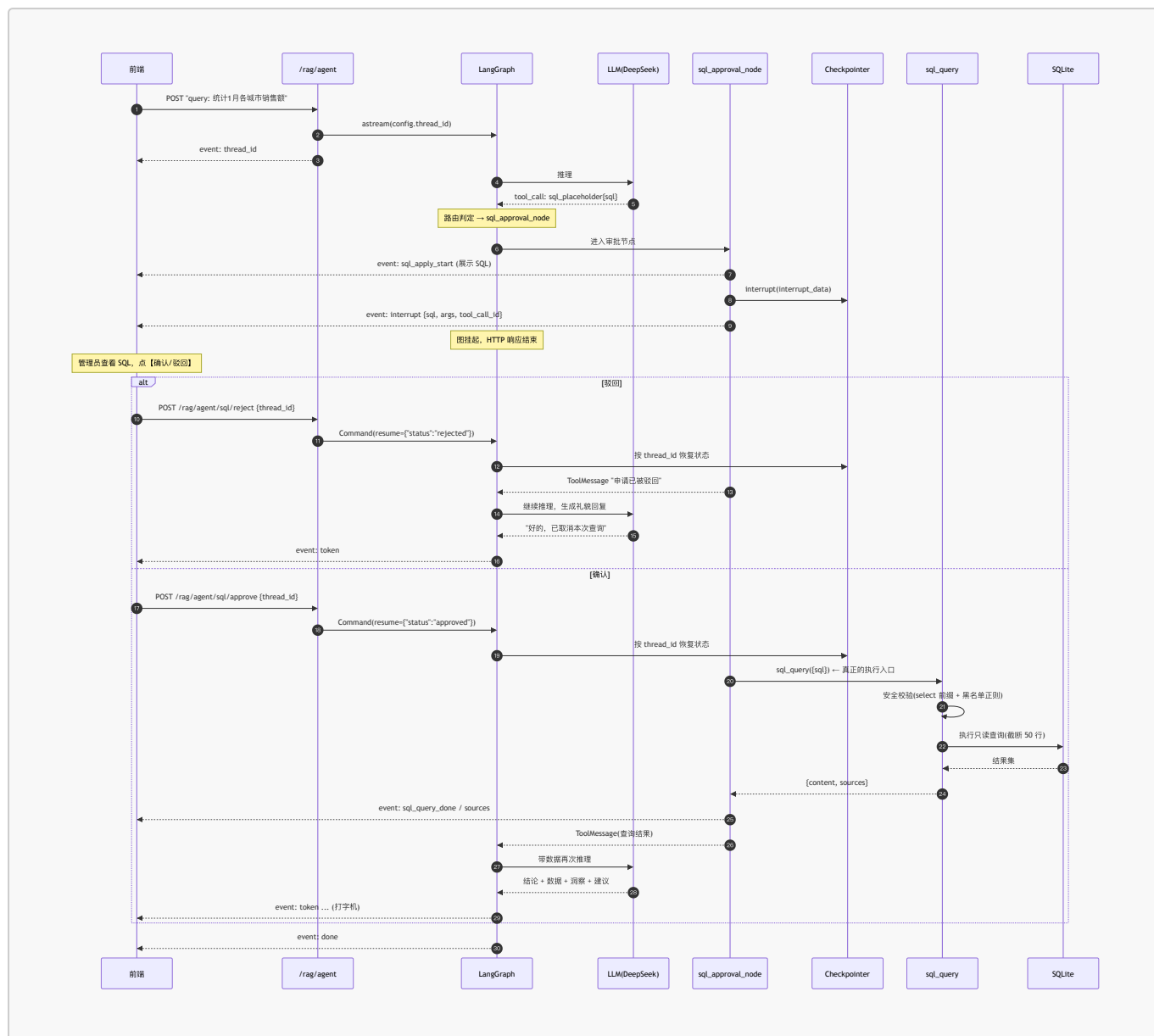
- 工具列表里放的是 **sql_placeholder**，函数体是 **lambda sql: None**，永不执行
- 真正的 **sql_query** 是普通 Python 函数，由**审批节点**在人工确认后调用，模型接触不到
- 挂起靠 **interrupt()**，恢复靠 **thread_id + Command(resume=...)**，跨 HTTP 请求恢复图状态

第四段：亮点收尾（30s）

3.1 文档问答 (RAG, 含工具回环)



3.2 数据库查询 + 人工审批（重点讲这条）



讲述要点：

- 恢复执行的是同一个图的同一个挂起点，靠 `thread_id` 定位
- 驳回不会中断对话，模型会收到"驳回"的 `ToolMessage` 并继续生成回复
- `sql_query` 不在 `bind_tools` 里，模型没有任何路径直接触发它

```
StructuredTool.from_function(
    lambda sql: None, # 永远不会执行, 仅占位
    name="sql_placeholder",
```

4. 技术选型与理由

选型	为什么选它	当时考虑的备选
LangGraph	需要条件路由（区分普通工具/审批）、回环（工具→LLM）、 <code>interrupt</code> 挂起恢复	LangChain AgentExecutor（路由与中断能力弱）；自己写状态机（成本高）
FastAPI	原生支持 <code>StreamingResponse</code> ，异步性能好，Pydantic 校验省事	Flask（流式与异步支持弱）
Chroma	轻量、本地持久化、LangChain 集成好，适合中小知识库	FAISS（无元数据管理）；Milvus（运维重）
智谱 embedding-2	中文语义效果好、国内直连稳定	bge-small-zh（本地部署需 ONNX 运行时，依赖重）；OpenAI embedding（网络与成本）
DeepSeek	中文推理强、成本低、兼容 OpenAI 协议可直接用 <code>ChatOpenAI</code>	通义/智谱 GLM
SQLite	演示/中小数据量够用，零运维	MySQL/Postgres（本项目阶段过重）
Tavily	专为 LLM 设计，返回 AI 总结答案，减少上下文占用	直接爬百度（反爬与解析成本高，项目里 <code>test_baidu.py</code> 验证过不可靠）

5. 四大亮点详解（展开讲）

亮点 1：把人工审批从"软约束"做成"硬隔离"

常规做法：Prompt 里写"执行 SQL 前必须征求用户同意"——模型可能不听。

我的做法：

- 给模型的 `sql_placeholder` 函数体是 `lambda sql: None`，永不执行
- 真实执行函数 `sql_query` 是普通 Python 函数，不进 `bind_tools`
- 审批通过后由节点调用它

这样模型能力上就没有直接查库的路径，审批成为结构性保证，而非概率性保证。

亮点 2：跨 HTTP 请求恢复图状态

审批可能持续几分钟，但 HTTP 是无状态的。解法：`checkpointner` 在 `interrupt` 处保存快照 → 前端拿 `thread_id` 发第二个请求 → `Command(resume=...)` 精确恢复。这是把"长事务"跑在"无状态服务"上的通用模式，也可用于其他需要人工介入的场景（如敏感操作二次确认）。

亮点 3：过程可观测，不是黑盒

用 `get_stream_writer()` 推自定义事件，前端按类型渲染：

事件	用途
<code>thread_id</code>	会话标识，供审批回传
<code>token</code>	打字机效果

事件	用途
tool_call	展示"正在调用 XX 工具，参数..."
sources	引用了哪几份文档
interrupt	弹出审批卡片
error / done	异常与结束

用户能看见 AI 在做什么，显著提升信任感，也方便排障。

亮点 4：三层记忆 + 动态 Prompt

层	实现	解决什么
短期	InMemorySaver + thread_id	多轮上下文、中断恢复
结构化画像	JSON 增量合并（改名时清空旧偏好）	记住"我叫什么、喜欢什么"
长期模糊记忆	LLM 摘要 → 向量化 → 检索	记住任意历史信息

工程细节：

- 写入前 similarity_search 判重，避免重复记忆堆积
- 检索时取 k*2 候选再按时间戳倒序取 top3，保证"最近的记忆优先"
- 每次请求动态拼装 当前日期 + 画像 + 记忆 注入 System Prompt，解决"上个月/今年"这类相对时间理解

6. 高频追问 Q&A

Q1：为什么不用 ReAct 或现成 Agent，要自己建图？ 因为需要条件路由把"SQL 申请"分流到审批节点，还要在审批处挂起。标准 AgentExecutor 的路由是固定 "LLM→工具→LLM"，无法插入人工审批与中断恢复。自建图才拿得到这个控制力。

Q2：怎么防止模型生成危险 SQL？ 四道防线：① Prompt 里写死只读规则与表结构；② 只允许 select 前缀；③ 黑名单正则（insert|update|delete|drop|alter|create|truncate，要求词边界匹配）；④ 结果截断 50 行防上下文爆炸。最外层还有人工审批兜底。

Q3：RAG 怎么控制幻觉？ Prompt 明确要求"文档未提及就说未提及，不要编造"；检索结果带 sources 一并下发给前端，答案可溯源；检索未命中时返回标准文案而不是让模型自由发挥。

Q4：切片参数 500/50 怎么定的？ 500 字符约等于中文 250~300 字，语义完整又不至于稀释向量；50 字符重叠避免跨块的句子被切断。分隔符针对中文做了定制：["\n\n","\n","。","! ","? ","; ","", ",", "",""]。

Q5：检索 k 为什么取 3？ 权衡召回率与上下文长度：3 条片段约 1500 字符，信息密度足够又不会挤占生成空间；记忆检索则取 k*2=6 条候选再按时间排序取 3，给排序留余量。

Q6：无限工具调用循环怎么防？ recursion_limit=10 限制最大步数；Prompt 里也要求"不要重复搜索"。超限时 LangGraph 抛 GraphRecursionError，被 except Exception 捕获后以 error 事件下发。

- Q7: interrupt 抛的 GraphInterrupt 怎么处理的？** /rag/agent 里 except GraphInterrupt 单独捕获，转成 interrupt 事件正常下发，不当作错误。另外节点内也主动推了一次 __interrupt__ 自定义事件做双保险。
- Q8: 计算器用 eval 安全吗？** 当前做了两道校验（Pydantic 白名单正则 + 函数内字符白名单），能挡住常见注入。但 eval 本质仍是风险点，生产环境应替换为 ast 表达式解析或专用计算库。
- Q9: 多用户怎么隔离？** 画像按 user_id 分文件，长期记忆按 user_id 建独立 Chroma collection，会话按 thread_id 隔离。目前 user_id 是硬编码常量（单用户验证阶段），接入登录态后从 token 解析即可。
- Q10: 服务重启后未完成的审批还能恢复吗？** 当前用的是 InMemorySaver，不能。这是已知限制，演示阶段够用；生产需要换成 SqliteSaver/RedisSaver 做持久化检查点。
- Q11: 删除文件后知识库干净吗？** 目前 delete_file 只清理了 data/file_cache.json 元数据，rag_engine.delete_file_from_store 已实现但未接线，向量库会残留——这是待完善的点。
- Q12: Embedding 为什么要换掉最初的 HuggingFace 本地模型？** 本地 bge-small-zh 需要 ONNX 运行时，部署包大、冷启动慢；换智谱 embedding-2 后中文效果更好且免运维，代价是多一次网络调用——对知识库写入（低频）可接受。
- Q13: 工具执行失败会中断整个对话吗？** 不会。工具内部 try/except 后返回统一结构 {"content": "知识库检索失败: ...", "sources": []}，图照常把"失败信息"交给 LLM，由模型决定如何向用户解释。外部还包了重试装饰器。
- Q14: 重试策略怎么设计的？** 指数退避 + ±20% 随机抖动（避免惊群），带 max_delay 上限。web_search 针对 TimeoutException/ConnectError 重试 3 次；知识库检索重试 2 次、退避更激进（0.5s 起）。
- Q15: 这个项目的最大价值是什么？** 不是"接了个大模型"，而是在模型能力与安全边界之间做了工程化的隔离设计——让 AI 能干活，但干不了的、危险的，交给人类决策，且这个过程对用户完全透明。

7. 已知不足与演进方向（主动说，体现反思）

现状	演进方向
InMemorySaver，重启丢状态	换 SqliteSaver/RedisSaver，支持生产级审批会话
user_id 硬编码	接入登录态，从 JWT 解析；thread_id 支持多会话
删除文件未清向量库	接线 delete_file_from_store，并做事务化保证一致性
calculator 使用 eval	改 ast 表达式解析
画像异步更新函数未接线	在 /rag/agent 结束后挂 background_tasks 触发
全项目大量 print 调试	统一收敛到 logger，按环境控制级别
单进程内存缓存文件元数据	迁 Redis / 数据库，支持多实例部署

8. 讲述 Tips

- 先说问题再说方案：不要一上来报技术栈，先说"为什么需要审批"。
- 核心亮点讲透一个：人工审批那条链路讲到位，比罗列十个功能更有说服力。

3. **主动暴露不足**：讲到"已知限制"时主动说出并给出改进方案，比被追问再承认好得多。
4. **准备一个"踩坑故事"**：例如最初用百度爬虫做搜索（见 `test_baidu.py`）反爬严重，换成 Tavily 后稳定且自带 AI 总结——体现调研与选型过程。
5. **量化表达**：切片 500/50、检索 Top3、结果截断 50 行、`recursion_limit=10`、100 条测试订单——具体数字让讲述更可信。

提示：本文档含 Mermaid 图，建议用 Typora / 飞书文档 / GitHub / VS Code(Mermaid 插件) 打开渲染；打印前可先导出 PDF。